RHYTHMGUARD
Behavioral Security and Fraud-Prevention Sandbox
Continuous Authentication
from Interaction Rhythm
End-to-end technical project report

Prepared for academic and technical review
August 2026

Use only with informed consent, documented retention, access, and deletion policies.

Executive Summary

RhythmGuard is an end-to-end behavioral biometric security system for a BFSI (banking, financial services, and insurance) fraud-prevention sandbox. Instead of relying only on passwords, device identifiers, or a one-time login decision, the system observes how a consenting user interacts with a browser. It converts typing rhythm and pointer movement into four numerical signals, evaluates rolling windows with a two-layer long short-term memory (LSTM) model, and routes risk decisions through a distributed streaming architecture.

The project is designed to detect behavioral changes associated with automated bots, synthetic identities, account takeover, and social-engineering or “duress signature” scenarios. It is not intended to identify a person from raw keystrokes, and the browser does not store the text typed by the user. The client extracts timing and motion features locally, sends bounded micro-batches to a FastAPI gateway, and the gateway places them on Kafka without waiting for model inference. A worker consumes the stream, retrieves a historical baseline from Redis, scores the latest 20-row window with PyTorch, and applies a policy: continue, request step-up MFA, or block a session.

The implemented system includes a modern capture interface, an authenticated operations dashboard, WebSocket alerting, Redis audit history, baseline comparison, bot simulation, infrastructure health, explainable alert factors, and a model-evaluation page. The current real-data evaluation is intentionally reported honestly: with six normal sessions and five bot sessions, the test accuracy was approximately 71.49%, recall 99.40%, F1 83.21%, and ROC-AUC 46.12%. The low ROC-AUC indicates that this experiment is a working research prototype, not a production-ready fraud classifier. More representative, balanced, consented data and better calibration are required before operational use.

Keywords: behavioral biometrics, continuous authentication, LSTM, anomaly detection, fraud risk scoring, Kafka, Redis, FastAPI, PyTorch, BFSI security.

Introduction and Motivation

Problem Context

Traditional authentication answers the question “did the user provide the correct credential?” It does not continuously answer “does the current interaction still resemble the trusted session?” A criminal may obtain a password through phishing, a synthetic identity may pass an onboarding check, or an automation tool may imitate the HTTP requests made by a browser. In these cases, interaction behavior can provide an additional, passive signal.

Financial systems also need a graduated response. A weak anomaly should not immediately terminate a legitimate customer session, while a strong signal should not be ignored. RhythmGuard therefore treats the model output as a risk score and maps it to a response policy:

RhythmGuard response policy.

Risk range	Status	System action
$0.00 \le s < 0.40$	Normal	Continue the session and write a low-risk audit event.
$0.40 \le s \le 0.70$	Suspicious	Set a temporary step-up MFA flag and notify the operations dashboard.
$s > 0.70$	Critical	Set a temporary block flag, delete a session token if present, and publish a critical alert.

Aim

The aim is to build a consent-aware, privacy-conscious, distributed behavioral security prototype that captures interaction telemetry, learns sequence patterns, scores live sessions, and supports analyst response.

Objectives

1. Capture high-resolution keyboard and pointer timing using without recording the typed text.

2. Derive dwell time, flight time, mouse velocity, and mouse acceleration in the browser.
3. Package features as bounded JSON micro-batches and return HTTP 202 quickly from the ingestion gateway.
4. Train and evaluate a two-layer, 64-hidden-unit LSTM over windows of shape $[B, 20, 4]$.
5. Personalize inference using a Redis historical baseline for a session.
6. Separate ingestion from inference using Kafka so the API and model worker can scale independently.
7. Provide explainable risk factors, alert history, authenticated response controls, audit logging, and monitoring.
8. Document limitations and the evidence required before deployment in a real financial system.

Scope and Non-Goals

The system is a research and demonstration sandbox. It does not replace a bank's identity provider, fraud case-management system, legal review, or accessibility testing. It does not store raw passwords or typed text. The demo admin password is only a local development convenience and must be replaced with a real identity and access-management system in production.

System Architecture

Logical Architecture

The architecture is intentionally decoupled. The browser performs inexpensive feature extraction; the gateway validates and queues payloads; Kafka provides a durable stream boundary; the worker performs Redis lookup and PyTorch inference; and the decision engine publishes alerts and enforcement flags. MongoDB or PostgreSQL can be added as a cold audit store, but the current local implementation uses Redis Streams for audit history.

RhythmGuard end-to-end architecture.

Execution Phases

1. **Phase 1 – Capture and feature extraction:** A consent-gated React page attaches , , and listeners. It derives four features locally and emits bounded micro-batches.

2. **Phase 2 – Machine learning:** Labeled normal and bot session files are validated, split by complete session, normalized against the normal training profile, and used to train/evaluate the LSTM.

3. **Phase 3 – Distributed ingestion and inference:** FastAPI publishes accepted payloads to Kafka. The worker consumes them, reads Redis baselines, constructs a 20-row window, and calculates risk.

4. **Phase 4 – Mitigation and operations:** The decision engine writes flags, stores audit events, publishes alerts, and exposes live operations, session timelines, model evaluation, and response controls.

Technology Stack and Requirements

Technology choices and responsibilitie

Layer	Technologies	Responsib
Client	React, Vite, JavaScript, CSS	Consent, capture, lo feature extraction, dataset exp live sessio presentatic
Gateway	Python, FastAPI, Pydantic	Schema validation, bounded n blocking enqueue, HTTP 202 response, health and control AP
Streaming	Apache Kafka, aiokafka	Decouple producers inference consumers through .
State and audit	Redis, Redis Streams, Pub/Sub	Profiles, la risk state, temporary flags, alert heartbeats, audit histo
ML	PyTorch, scikit-learn	Two-layer LSTM, normalizat training, threshold

Layer	Technologies	Responsib
		selection, metrics.
Operations	WebSocket, HTML/CSS UI	Live alert feed, risk distributio service hea timeline, operator actions.
Runtime	Docker Compose, Uvicorn	Local Kafka/Red infrastruct and Pytho API proce

Data Collection and Feature Engineering

Keyboard Features

For each non-repeated key, the collector stores the keydown timestamp. On keyup it computes dwell time:

$$d_i = t_i^{keyup} - t_i^{keydown}.$$

The collector also keeps the preceding keyup timestamp. At the keydown of the current key it computes flight time:

$$f_i = t_i^{keydown} - t_{i-1}^{keyup}.$$

The first flight time is set to zero because no previous keyup exists. Matching uses where possible, which is more stable across keyboard layouts than the displayed character.

Pointer Features

For consecutive pointer coordinates (x_{i-1}, y_{i-1}) and (x_i, y_i), the Euclidean distance and elapsed time are:

$$\begin{equation} \Delta_i = \sqrt{(x_i-x_{i-1})^2+(y_i-y_{i-1})^2}, \qquad \delta t_i = t_i-t_{i-1}. \end{equation}$$

Velocity is distance per elapsed time and acceleration is the velocity change per elapsed time:

$$\begin{equation} v_i = \frac{\Delta_i}{\delta t_i}, \qquad a_i = \frac{v_i-v_{i-1}}{\delta t_i}. \end{equation}$$

The row contract is therefore:

$$x_i = [d_i, f_i, v_i, a_i].$$

Privacy and Non-Blocking Behavior

The browser stores timing and motion features, not typed characters. Event handlers use mutable buffers and schedule batch work with a microtask or timer so React state updates and network operations do not occur directly inside high-frequency pointer handlers. A batch is bounded at 20 rows and sent with . The capture screen requires an explicit consent checkbox and exposes local JSON export and local-data deletion.

JSON Contract

```
{
  "session_id": "a-session-id",
  "sequence": 12,
  "client_timestamp_ms": 1710000000000,
  "event_types": ["keyboard", "mouse"],
  "telemetry": [
    [12.4, 83.1, 0.0, 0.0],
    [0.0, 0.0, 0.42, 0.03]
  ]
}
```

The API validates that each row has exactly four finite features. Dwell, flight, and velocity must be non-negative; acceleration may be signed because negative acceleration represents deceleration.

Dataset Construction and Validation

The project includes consented/locally captured normal and bot-labeled JSON sessions plus a clearly separated synthetic dataset for pipeline wiring. The validator checks filename-label agreement, declared label, session ID, row count, complete 20-row windows, and duplicate session IDs.

Recorded real-data validation summary.

Item	Value
Normal sessions	6
Bot sessions	5
Total telemetry rows	37,224
Complete 20-row windows	37,015
Duplicate session IDs	0
Ready for training	Yes

The evaluator splits complete session files rather than individual rows. This is essential: if windows from one session appear in both training and test data, the model may memorize a session-specific pattern and produce an over-optimistic result. The current split contains four normal and three bot files for training, one of each for validation, and one of each for test.

Machine Learning Model

Input and Normalization

The worker constructs exactly the latest 20 rows. If fewer than 20 are available, it left-pads with the session baseline mean. For the normal training split, a profile mean μ and standard deviation σ are calculated. Each feature is normalized as:

$$\begin{equation} z_{t,j} = \frac{x_{t,j}-\mu_j}{\max(\sigma_j,\epsilon)}. \end{equation}$$

The model input shape is $[B, 20, 4]$, where B is batch size, 20 is the sequence length, and 4 is the number of features.

BioSentinelLSTM Architecture

Although the internal compatibility class is named , the product is called RhythmGuard. The network has:

1. a two-layer LSTM with input size 4 and hidden size 64;
2. dropout of 0.1 between recurrent layers;
3. the final hidden state from the second layer;
4. a fully connected layer from 64 to 1;
5. a sigmoid activation producing a score in $[0, 1]$.

The training placeholder uses Adam with learning rate 10^{-3} and binary cross-entropy loss. A weighted sampler balances normal and bot windows in the training loader. Validation and test sets are not oversampled.

Threshold Tuning

The validation split chooses the threshold with the best F1 score, preferring precision and then the higher threshold in ties. Test labels are not used for threshold selection. The recorded threshold was approximately 0.4593.

Evaluation Results

Recorded Metrics

Leakage-safe evaluation results from
.

Metric	Validation	Test
Samples	119	235
Threshold	0.4593	0.4593
Accuracy	95.80%	71.49%
Precision	95.80%	71.55%
Recall	100.00%	99.40%
F1 score	97.85%	83.21%
ROC-AUC	35.96%	46.12%

Test Confusion Matrix

The test confusion matrix is ordered as actual rows and predicted columns, with normal as class 0 and bot as class 1:

Test confusion matrix.

	Predicted normal	Predicted bot
Actual normal	2	66
Actual bot	1	166

The model catches nearly all bot windows in this test split, but it also raises many false positives for normal windows. The ROC-AUC below 0.5 is a warning that the score ranking is not reliable across this small and imbalanced experiment. The result should be used to demonstrate the evaluation pipeline and motivate more data collection, calibration, and model comparison—not to claim production fraud-detection performance.

API Gateway and Streaming Worker

FastAPI Gateway

The endpoint accepts either one JSON envelope or a JSON array of envelopes. Pydantic checks the session ID, row count, feature count, finite values, and non-negative fields. After validation, the gateway calls a bounded method. If the queue is not full it

immediately returns HTTP 202 with the accepted count, queue depth, and topic. Kafka network delivery happens in a background sender task, so the request does not wait for inference.

Kafka Stream

The topic is . The browser and API do not need to know which worker instance will process a message. Kafka provides the decoupling point for horizontal workers, replay, consumer groups, and back-pressure monitoring. The current worker uses manual commits: a message is committed after profile lookup, inference, decision handling, and alert/audit persistence succeed.

Redis Profile Lookup

For a session ID *s*, the worker reads:

.

A profile contains four means and four standard deviations. A Redis profile overrides the checkpoint profile for per-user personalization. If Redis has no profile, the worker uses statistics stored in the model checkpoint. The latest result is stored under ; temporary flags use and .

Explainability

The worker computes baseline-relative evidence in addition to the neural score. It reports features whose mean absolute standardized deviation is high and detects unusually regular dwell or flight times. These are supporting indicators, not causal proof of fraud. The dashboard presents them as “evidence to inspect” rather than as a definitive identity claim.

Decision Engine and Real-Time Operations

The decision engine centralizes action policy. It records normal events in the audit stream, publishes suspicious and critical events to Redis Pub/Sub and a capped alert history stream, and updates the latest session result. The FastAPI WebSocket endpoint subscribes to and forwards new events to the operations page.

The admin console provides:

- WebSocket connection state and live alert counts;
- Kafka, Redis, API, worker-heartbeat, and queue-depth health;
- risk-distribution bar for critical, suspicious, and other events;

- explainable factors on each alert;
- session timeline from alerts and audit streams;
- authenticated Require MFA, Block, Restore, and Acknowledge controls;
- Redis-backed operator/system audit history.

The local login endpoint issues a short-lived in-memory bearer token. The default password is only when is not set. A real deployment must use an external identity provider, strong secret management, role-based access, and preferably dual approval for blocking a financial session.

User Interface and Demonstration Flow

Capture Lab

The capture screen is a consent-gated behavioral sandbox. The user can type naturally, move inside the pointer field, watch batch/sample counters, export a labeled JSON file, clear local data, and run a clearly marked 20-row bot simulation. The readiness meter reports how close the session is to a recommended 20-row model window. It does not claim that a user is trustworthy; it reports collection readiness.

Model Center

The model center reads the evaluation JSON artifact and displays test metrics, the confusion matrix, threshold, split sizes, and a validation check. If the report is missing, the page says so instead of inventing performance values.

Typical Demonstration

1. Start Kafka and Redis with Docker Compose.
2. Start the FastAPI gateway and the worker.
3. Open the capture lab and grant demo consent.
4. Type and move the pointer until a window is sent.
5. Observe the 202 gateway state and the live worker score.
6. Use the bot simulation to send regularized timing features.
7. Open the operations console to inspect the alert, explanation, timeline, and response policy.
8. Open the model center to discuss test metrics and limitations.

Installation and Runbook

Prerequisites

Python 3.11+ is recommended, Node.js with npm is required for the frontend, and Docker Desktop is required for local Kafka/Redis. The Python dependencies are listed in .

Commands

```
docker compose up -d
python -m pip install -r requirements.txt

# Terminal 1: gateway
uvicorn api.main:app --reload --port 8000

# Terminal 2: streaming worker
$env:RHYTHMGUARD_MODEL_CHECKPOINT="artifacts\rhythmguard_lstm.pt"
python -m worker.main

# Terminal 3: frontend
cd frontend
npm install
npm run dev
```

Useful URLs

URL	Purpose
	Consent-gated capture lab.
	Live operations, controls, and audit view.
	Model metrics and confusion matrix.
	FastAPI interactive API documentation.
	Infrastructure health JSON.

Training and Evaluation Commands

```
python -m ml.validate_dataset --normal .\datasets\normal_*.json --bot
        .\datasets\bot_*.json
```

```
python -m ml.evaluate \
  --normal .\datasets\normal_1.json .\datasets\normal_2.json
        .\datasets\normal_3.json \
  --bot .\datasets\bot_1.json .\datasets\bot_2.json .\datasets\bot_3.json \
  --epochs 10 \
  --output artifacts\rhythmguard_evaluated.pt \
  --report artifacts\evaluation_report.json
```

Outputs and Evidence

The project produces the following artifacts and runtime outputs:

Important project outputs.

Artifact	Meaning
	Labeled session telemetry with four-feature rows.
	Data-quality and complete-window checks.
	Split definition, metrics, threshold, confusion matrices, and profile.
	Runtime checkpoint used by the worker.
	Capped Redis history for suspicious/critical alerts.
	Redis Stream for system and operator events.
	Latest score, action, factors, and reason for a session.

The current implementation was verified with a successful frontend production build, Python compilation, and the project test suite. The latest test run completed with three passing tests. Live smoke tests returned HTTP 200 for system health, model evaluation, risk lookup, authenticated audit access, and session timeline access.

Security, Privacy, and Ethical Considerations

Behavioral telemetry can be sensitive and may qualify as biometric or personal data depending on jurisdiction and intended use. The following controls are implemented or required:

- **Consent:** The capture collector remains paused until the user checks the consent box.
- **Data minimization:** The feature contract uses timing and pointer dynamics, not typed text or password values.
- **Retention:** The demo exposes local deletion; production retention periods must be defined and enforced server-side.
- **Access control:** The local admin control channel is authenticated; production requires an identity provider, least privilege, and audit review.
- **Human oversight:** A risk score is decision support. It should not be the sole basis for denial of financial service.
- **Fairness and accessibility:** Motor impairments, assistive technologies, keyboard layouts, age, stress, device differences, and network conditions may alter behavior. The system must be tested for disparate impact and must provide alternative authentication paths.
- **Adversarial robustness:** Attackers may deliberately imitate a baseline. Model outputs should be combined with device, transaction, identity, and fraud-network signals rather than used alone.

Limitations

1. The real evaluation contains only six normal and five bot sessions, with substantial window imbalance.
2. The test ROC-AUC of 0.4612 indicates poor ranking quality and prevents a production-performance claim.
3. The bot label is a proxy for scripted or regular behavior, not proof that every flagged session is malicious.
4. The local admin token store is in process memory and is not a production authentication system.

5. Redis flags are demonstrated enforcement hooks; integration with the real token/session service is still required.
6. A single LSTM and four features cannot represent the full context of a financial fraud event.
7. No false-positive cost model, calibration curve, population stability index, or long-term drift study is included yet.
8. The demo does not provide a full analyst case-management workflow or database-backed immutable audit storage.

Future Work

Immediate Engineering Work

- Collect more consented sessions across users, devices, keyboard layouts, and realistic bot strategies.
- Rebalance and stratify evaluation at the session level; report confidence intervals and repeated seeds.
- Compare the LSTM with logistic regression, random forest, isolation forest, one-class SVM, temporal convolution, and transformer baselines.
- Calibrate scores and tune thresholds using business costs for false positives and false negatives.
- Add a real database for immutable audit records and a proper case-management workflow.
- Add Kafka lag, throughput, latency percentiles, retry counts, and worker autoscaling metrics.

Security and Product Work

- Replace the local admin password with OIDC/SAML, RBAC, MFA, short-lived signed tokens, CSRF protection, and dual approval for blocking.
- Encrypt data in transit and at rest, use managed secrets, rotate keys, and implement retention/deletion jobs.
- Add a profile enrollment flow with baseline quality checks and drift-aware retraining.
- Combine behavior with transaction velocity, device reputation, IP/ASN risk, impossible travel, and identity-verification signals.
- Add adversarial testing for replay, timing injection, browser automation, and model evasion.

- Provide a transparent user notice, appeal path, and non-biometric fallback authentication route.

Conclusion

RhythmGuard demonstrates how a simple browser interaction signal can be transformed into a distributed security workflow. The browser computes privacy-conscious timing and motion features; FastAPI accepts them without blocking on inference; Kafka separates ingestion from computation; Redis supplies personalization and enforcement state; PyTorch evaluates a fixed sequence; and the decision engine turns the score into an explainable, auditable response.

The project is valuable as an engineering and research foundation because it makes the full chain visible: data contract, feature mathematics, model shape, evaluation discipline, stream processing, operational UI, and governance concerns. Its current metrics also show why honest evaluation matters. The system is wired end-to-end, but the measured ROC-AUC and small dataset mean that it must not be presented as a finished fraud detector. The appropriate next step is more representative data, stronger evaluation, calibrated thresholds, and production-grade privacy and access controls.

How to Explain the Project in One Minute

> RhythmGuard is a continuous-authentication prototype for BFSI fraud prevention. With consent, it measures how a person types and moves a pointer without storing the typed text. The browser converts those actions into four timing features and sends small batches to FastAPI. Kafka carries the stream to a worker, Redis provides the user's historical baseline, and a PyTorch LSTM scores the latest 20 events. The decision engine continues normal sessions, requests MFA for suspicious behavior, and blocks critical sessions. An operations dashboard shows live alerts, reasons, timelines, health, and audit records. The current result proves the end-to-end architecture, while the evaluation also shows that more real data is required before production use.

Project Directory Map

```
RhythmGuard/
|-- api/
|   |-- main.py                # FastAPI gateway and security endpoints
|   |-- schemas.py             # Pydantic telemetry/admin contracts
|   `-- kafka_publisher.py     # bounded async Kafka handoff
|-- worker/
|   |-- main.py                # Kafka consumer, Redis profile, inference
|   `-- decision_engine.py     # thresholds, flags, alerts, audit
```

```
|-- ml/
|   |-- model.py                # two-layer 64-unit BioSentinelLSTM
|   |-- dataset.py              # JSON loading and profiles
|   |-- train.py                # training entry point
|   |-- evaluate.py             # session split, metrics, threshold tuning
|   `-- validate_dataset.py      # dataset quality checks
|-- frontend/src/
|   |-- App.jsx                 # capture lab
|   |-- AdminDashboard.jsx      # operations and response controls
|   |-- EvaluationDashboard.jsx # model center
|   |-- telemetryCapture.js     # non-blocking browser extractor
|   `-- styles.css              # interface and security theme
|-- datasets/                   # normal and bot JSON sessions
|-- artifacts/                 # checkpoints, reports, validation outputs
|-- scripts/stress_test.py      # gateway load test
|-- docker-compose.yml          # local Kafka and Redis
`-- requirements.txt
```

Core Source Code Appendix

The following listings are included so the report contains the principal implementation code. The repository remains the authoritative, complete source because the React interface and style sheet are intentionally large.

Browser Feature Extraction

```
/**
 * Browser-side biometric feature extraction.
 *
 * A row always has this shape:
 * [dwellTimeMs, flightTimeMs, mouseVelocityPxPerMs,
 *       mouseAccelerationPxPerMs2]
 *
 * The collector intentionally does not call React state setters from event
 * handlers. It keeps a small mutable buffer and emits batches on a timer or
 * when the batch size is reached.
 */

export const TELEMETRY_FEATURES = 4;

const finiteNonNegative = (value) =>
  Number.isFinite(value) && value >= 0 ? value : 0;

const finite = (value) => (Number.isFinite(value) ? value : 0);

const scheduleMicrotask = (callback) => {
  if (typeof queueMicrotask === "function") {
    queueMicrotask(callback);
```

```
    } else {
      Promise.resolve().then(callback);
    }
  };

  export class TelemetryCollector {
    constructor({
      sessionId,
      onBatch,
      batchSize = 20,
      flushIntervalMs = 1000,
      keyboardTarget = null,
      mouseTarget = null,
    }) {
      if (!sessionId) throw new Error("sessionId is required");
      if (typeof onBatch !== "function") throw new Error("onBatch is
           required");

      this.sessionId = sessionId;
      this.onBatch = onBatch;
      this.batchSize = Math.max(1, Math.floor(batchSize));
      this.flushIntervalMs = Math.max(50, Math.floor(flushIntervalMs));
      this.keyboardTarget =
        keyboardTarget ?? (typeof document !== "undefined" ? document : null);
      this.mouseTarget = mouseTarget;

      this.started = false;
      this.pendingRows = [];
      this.keyDowns = new Map();
      this.lastKeyUpTime = null;
      this.lastMousePoint = null;
      this.sequence = 0;
      this.flushTimer = null;
      this.flushScheduled = false;

      this.handleKeyDown = this.handleKeyDown.bind(this);
      this.handleKeyUp = this.handleKeyUp.bind(this);
      this.handleMouseMove = this.handleMouseMove.bind(this);
    }

    start() {
      if (this.started) return;
      this.started = true;
      this.keyboardTarget?.addEventListener("keydown", this.handleKeyDown, {
        passive: true,
      });
      this.keyboardTarget?.addEventListener("keyup", this.handleKeyUp, {
        passive: true,
      });
      this.mouseTarget?.addEventListener("mousemove", this.handleMouseMove, {
        passive: true,
```

```
    });
    this.flushTimer = globalThis.setInterval(() => this.flush(),
        this.flushIntervalMs);
  }

  stop() {
    if (!this.started) return;
    this.started = false;
    this.keyboardTarget?.removeEventListener("keydown", this.handleKeyDown);
    this.keyboardTarget?.removeEventListener("keyup", this.handleKeyUp);
    this.mouseTarget?.removeEventListener("mousemove", this.handleMouseMove);
    if (this.flushTimer !== null) globalThis.clearInterval(this.flushTimer);
    this.flushTimer = null;
    this.keyDowns.clear();
    this.lastMousePoint = null;
    this.flush();
  }

  handleKeyDown(event) {
    if (!this.started || event.repeat) return;

    // code is layout-independent and lets keydown/keyup match reliably.
    const keyId = event.code || event.key;
    if (!keyId || this.keyDowns.has(keyId)) return;

    const timestamp = performance.now();
    const flightTime =
      this.lastKeyUpTime === null
        ? 0
        : finiteNonNegative(timestamp - this.lastKeyUpTime);

    this.keyDowns.set(keyId, { timestamp, flightTime });
  }

  handleKeyUp(event) {
    if (!this.started) return;

    const keyId = event.code || event.key;
    const keyState = this.keyDowns.get(keyId);
    if (!keyState) return;

    const timestamp = performance.now();
    const dwellTime = finiteNonNegative(timestamp - keyState.timestamp);

    // flightTime was captured at keydown as:
    // current keydown timestamp - previous keyup timestamp.
    this.enqueueRow(
      [dwellTime, keyState.flightTime, 0, 0],
      "keyboard",
      timestamp,
    );
```

```
    this.keyDowns.delete(keyId);
    this.lastKeyUpTime = timestamp;
  }

  handleMouseMove(event) {
    if (!this.started) return;

    const timestamp = performance.now();
    const point = { x: event.clientX, y: event.clientY, timestamp };
    if (this.lastMousePoint === null) {
      this.lastMousePoint = { ...point, velocity: 0 };
      return;
    }

    const deltaTime = timestamp - this.lastMousePoint.timestamp;
    if (!(deltaTime > 0)) return;

    const dx = point.x - this.lastMousePoint.x;
    const dy = point.y - this.lastMousePoint.y;
    const distance = Math.hypot(dx, dy);
    const velocity = finiteNonNegative(distance / deltaTime);
    // Acceleration is signed: a negative value represents deceleration.
    const acceleration = finite(
      (velocity - this.lastMousePoint.velocity) / deltaTime,
    );

    this.enqueueRow([0, 0, velocity, acceleration], "mouse", timestamp);
    this.lastMousePoint = { ...point, velocity };
  }

  enqueueRow(features, eventType, timestamp) {
    if (features.length !== TELEMETRY_FEATURES) return;
    this.pendingRows.push({
      features: [
        finiteNonNegative(features[0]),
        finiteNonNegative(features[1]),
        finiteNonNegative(features[2]),
        finite(features[3]),
      ],
      event_type: eventType,
      captured_at_ms: timestamp,
    });

    if (this.pendingRows.length >= this.batchSize) this.scheduleFlush();
  }

  scheduleFlush() {
    if (this.flushScheduled) return;
    this.flushScheduled = true;
    scheduleMicrotask(() => {
      this.flushScheduled = false;
```

```
      this.flush();
    });
  }

  flush() {
    if (this.pendingRows.length === 0) return;

    // Emit all currently available rows as one bounded micro-batch. A timer
    // flush can therefore send a partial batch, while a busy user reaches
        the
    // configured batch size without blocking the event handler.
    const rows = this.pendingRows.splice(0, this.batchSize);
    const batch = {
      session_id: this.sessionId,
      sequence: this.sequence++,
      client_timestamp_ms: Date.now(),
      telemetry: rows.map((row) => row.features),
      // Useful for local debugging; the model consumes only telemetry.
      event_types: rows.map((row) => row.event_type),
    };

    scheduleMicrotask(() => {
      try {
        const result = this.onBatch(batch);
        if (result && typeof result.catch === "function") result.catch(() =>
         {});
      } catch {
        // A telemetry sink must never break the typing or pointer
         experience.
      }
    });

    if (this.pendingRows.length > 0) this.scheduleFlush();
  }
}

export function createTelemetryCollector(options) {
  return new TelemetryCollector(options);
}
```

PyTorch Model

```
from pathlib import Path
from typing import Optional, Union

import torch
from torch import Tensor, nn


class BioSentinelLSTM(nn.Module):
```

```
    """Continuous-authentication model for 20 rows of four telemetry
        features."""

    sequence_length = 20
    feature_count = 4

    def __init__(self, input_size: int = 4, hidden_size: int = 64,
         num_layers: int = 2):
        super().__init__()
        if input_size != self.feature_count:
            raise ValueError("BioSentinelLSTM expects four input features")
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=0.1 if num_layers > 1 else 0.0,
        )
        self.classifier = nn.Linear(hidden_size, 1)
        self.activation = nn.Sigmoid()

    @staticmethod
    def _profile_tensor(value, batch_size: int, device: torch.device) ->
         Tensor:
        tensor = torch.as_tensor(value, dtype=torch.float32, device=device)
        if tensor.ndim == 1:
            tensor = tensor.unsqueeze(0)
        if tensor.ndim != 2 or tensor.shape[-1] !=
         BioSentinelLSTM.feature_count:
            raise ValueError("baseline profile values must have shape [4] or
         [batch_size, 4]")
        if tensor.shape[0] == 1:
            tensor = tensor.expand(batch_size, -1)
        if tensor.shape[0] != batch_size:
            raise ValueError("baseline profile batch size does not match
         input")
        return tensor

    def forward(
        self,
        sequence: Tensor,
        baseline_mean: Optional[Tensor] = None,
        baseline_std: Optional[Tensor] = None,
    ) -> Tensor:
        if sequence.ndim != 3:
            raise ValueError("input must have shape [batch_size,
         sequence_length, features]")
        if sequence.shape[1] != self.sequence_length or sequence.shape[2] !=
         self.input_size:
            raise ValueError("input must have shape [batch_size, 20, 4]")
```

```
        normalized = sequence.float()
        if baseline_mean is not None or baseline_std is not None:
            if baseline_mean is None or baseline_std is None:
                raise ValueError("baseline_mean and baseline_std must be
 supplied together")
            mean = self._profile_tensor(baseline_mean, sequence.shape[0],
 sequence.device)
            std = self._profile_tensor(baseline_std, sequence.shape[0],
 sequence.device).clamp_min(1e-6)
            normalized = (normalized - mean.unsqueeze(1)) / std.unsqueeze(1)

        _, (hidden, _) = self.lstm(normalized)
        logits = self.classifier(hidden[-1]).squeeze(-1)
        return self.activation(logits)


def load_model(
    checkpoint: Optional[Union[str, Path]] = None,
    device: Optional[Union[str, torch.device]] = None,
) -> BioSentinelLSTM:
    target_device = torch.device(device or ("cuda" if
         torch.cuda.is_available() else "cpu"))
    model = BioSentinelLSTM().to(target_device)
    if checkpoint:
        checkpoint_path = Path(checkpoint)
        if not checkpoint_path.exists():
            raise FileNotFoundError(f"model checkpoint does not exist:
 {checkpoint_path}")
        state = torch.load(checkpoint_path, map_location=target_device,
 weights_only=False)
        if isinstance(state, dict) and "model_state_dict" in state:
            profile = state.get("profile", {})
            if profile.get("mean") is not None and profile.get("std") is not
 None:
                model.profile_mean = [float(value) for value in
 profile["mean"]]
                model.profile_std = [float(value) for value in
 profile["std"]]
            state = state["model_state_dict"]
        model.load_state_dict(state)
    model.eval()
    return model


def training_loop_placeholder(model: BioSentinelLSTM, train_loader, epochs:
     int = 1):
    """Minimal training hook; replace the loader with consented labeled
     data."""
    criterion = nn.BCELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    model.train()
    for _ in range(epochs):
        for features, labels in train_loader:
```

```
            optimizer.zero_grad(set_to_none=True)
            scores = model(features)
            loss = criterion(scores, labels.float())
            loss.backward()
            optimizer.step()
    return model
```

FastAPI Gateway and Security Contracts

```
import asyncio
import os
import json
import secrets
import time
from contextlib import asynccontextmanager
from pathlib import Path
from typing import Any, Dict, List, Optional

from fastapi import FastAPI, Header, HTTPException, Request, WebSocket,
        WebSocketDisconnect, status

from .kafka_publisher import KafkaPublisher, publisher_from_env
from .schemas import (
    AcceptedResponse,
    AdminActionRequest,
    AdminLoginRequest,
    TelemetryEnvelope,
    TelemetryRequest,
)

try:
    import redis.asyncio as redis
except ImportError:  # pragma: no cover - dependency is installed in
        deployment
    redis = None


@asynccontextmanager
async def lifespan(app: FastAPI):
    publisher = publisher_from_env()
    await publisher.start()
    app.state.publisher = publisher
    app.state.admin_tokens = {}
    try:
        yield
    finally:
        await publisher.stop()


app = FastAPI(
```

```
    title="RhythmGuard Telemetry Gateway",
    version="1.0.0",
    lifespan=lifespan,
)


def as_dict(model):
    return model.model_dump() if hasattr(model, "model_dump") else
         model.dict()


async def get_publisher(request: Request) -> KafkaPublisher:
    """Support ASGI lifespan and lightweight direct test clients."""
    publisher = getattr(request.app.state, "publisher", None)
    if publisher is None:
        publisher = publisher_from_env()
        await publisher.start()
        request.app.state.publisher = publisher
    return publisher


@app.get("/health")
async def health(request: Request):
    publisher = await get_publisher(request)
    return {"status": "ok", "kafka_queue_depth": publisher.queue_depth}


FEATURE_NAMES = [
    "dwell_time_ms",
    "flight_time_ms",
    "mouse_velocity_px_per_ms",
    "mouse_acceleration_px_per_ms2",
]


def _redis_url() -> str:
    return os.getenv("REDIS_URL", "redis://localhost:6379/0")


async def _open_redis():
    if redis is None:
        return None
    return redis.from_url(_redis_url(), decode_responses=True)


async def _audit(client: Any, event: Dict[str, Any]) -> None:
    if client is None:
        return
    payload = {key: str(value) for key, value in event.items() if value is
         not None}
    await client.xadd("rhythmguard:audit", payload, maxlen=10_000,
         approximate=True)
```

```
def _token_is_valid(request: Request, authorization: Optional[str]) -> bool:
    if not authorization or not authorization.lower().startswith("bearer "):
        return False
    token = authorization.split(" ", 1)[1].strip()
    tokens = getattr(request.app.state, "admin_tokens", {})
    expiry = tokens.get(token)
    if expiry is None or expiry <= time.time():
        tokens.pop(token, None)
        return False
    return True


def _require_admin(request: Request, authorization: Optional[str]) -> None:
    if not _token_is_valid(request, authorization):
        raise HTTPException(status_code=401, detail="admin authentication
         required")


@app.post("/api/v1/admin/login")
async def admin_login(payload: AdminLoginRequest, request: Request):
    expected = os.getenv("RHYTHMGUARD_ADMIN_PASSWORD", "rhythmguard-demo")
    if not secrets.compare_digest(payload.password, expected):
        raise HTTPException(status_code=401, detail="invalid admin password")
    token = secrets.token_urlsafe(32)
    tokens = getattr(request.app.state, "admin_tokens", None)
    if tokens is None:
        tokens = {}
        request.app.state.admin_tokens = tokens
    tokens[token] = time.time() + 8 * 60 * 60
    client = await _open_redis()
    try:
        await _audit(client, {"event": "admin_login", "created_at_ms":
         int(time.time() * 1000)})
    finally:
        if client is not None:
            await client.aclose()
    return {"token": token, "expires_in_seconds": 8 * 60 * 60, "mode":
         "local-demo" if "RHYTHMGUARD_ADMIN_PASSWORD" not in os.environ else
         "configured"}


@app.get("/api/v1/system/health")
async def system_health(request: Request):
    publisher = await get_publisher(request)
    redis_state = "unavailable"
    worker_state = "unknown"
    worker_age_ms = None
    client = await _open_redis()
    try:
        if client is not None:
```

```
            try:
                await client.ping()
                redis_state = "healthy"
                heartbeat = await client.get("rhythmguard:worker:heartbeat")
                if heartbeat:
                    worker_age_ms = max(0, int(time.time() * 1000) -
     int(heartbeat))
                    worker_state = "healthy" if worker_age_ms < 30_000 else
     "stale"
                else:
                    worker_state = "waiting"
            except Exception:
                redis_state = "unreachable"
        finally:
            if client is not None:
                await client.aclose()
        return {
            "api": "healthy",
            "kafka": "disabled" if not publisher.enabled else ("connected" if
             publisher.producer is not None else "standby"),
            "kafka_queue_depth": publisher.queue_depth,
            "redis": redis_state,
            "worker": worker_state,
            "worker_age_ms": worker_age_ms,
            "checked_at_ms": int(time.time() * 1000),
        }


    @app.get("/api/v1/model/evaluation")
    async def model_evaluation():
        report_path = Path(os.getenv("RHYTHMGUARD_EVALUATION_REPORT",
             "artifacts/evaluation_report.json"))
        if not report_path.exists():
            return {"available": False, "message": "No evaluation report has been
             generated yet."}
        try:
            report = json.loads(report_path.read_text(encoding="utf-8"))
        except (OSError, json.JSONDecodeError) as exc:
            raise HTTPException(status_code=500, detail=f"could not read
             evaluation report: {exc}")
        return {"available": True, "report": report, "path": str(report_path)}


    def _decode_json(raw: Any) -> Optional[Dict[str, Any]]:
        if raw is None:
            return None
        try:
            value = json.loads(raw)
        except (TypeError, json.JSONDecodeError):
            return None
        return value if isinstance(value, dict) else None
```

```python
@app.get("/api/v1/sessions/{session_id}/risk")
async def session_risk(session_id: str):
    client = await _open_redis()
    if client is None:
        return {"session_id": session_id, "status": "unavailable", "score":
         None, "factors": []}
    try:
        current = _decode_json(await client.get(f"rhythmguard:session:
         {session_id}:latest"))
        if current is None:
            return {"session_id": session_id, "status": "warming_up",
         "score": None, "factors": []}
        current["session_id"] = session_id
        current["blocked"] = bool(await client.exists(f"session:blocked:
         {session_id}"))
        current["step_up_required"] = bool(await
         client.exists(f"session:step_up:{session_id}"))
        return current
    finally:
        await client.aclose()


@app.get("/api/v1/sessions/{session_id}/baseline")
async def session_baseline(session_id: str):
    client = await _open_redis()
    if client is None:
        return {"available": False, "feature_names": FEATURE_NAMES}
    try:
        profile = _decode_json(await client.get(f"user:profile:
         {session_id}"))
        if profile is None:
            return {"available": False, "feature_names": FEATURE_NAMES}
        baseline = profile.get("baseline", profile)
        means = baseline.get("mean", baseline.get("baseline_mean"))
        stds = baseline.get("std", baseline.get("baseline_std"))
        if not isinstance(means, list) or not isinstance(stds, list) or
         len(means) != 4 or len(stds) != 4:
            return {"available": False, "feature_names": FEATURE_NAMES}
        return {"available": True, "feature_names": FEATURE_NAMES, "mean":
         means, "std": stds}
    finally:
        await client.aclose()


@app.get("/api/v1/sessions/{session_id}/timeline")
async def session_timeline(session_id: str):
    client = await _open_redis()
    if client is None:
        return []
    events = []
    try:
```

```
            for stream_name in ("rhythmguard:alerts:history",
             "rhythmguard:audit"):
                entries = await client.xrevrange(stream_name, count=200)
                for entry_id, fields in entries:
                    event = _decode_json(fields.get("payload")) or dict(fields)
                    if str(event.get("session_id", "")) != session_id:
                        continue
                    event["stream_id"] = entry_id
                    event["created_at_ms"] = int(event.get("created_at_ms", 0) or
             0)
                    events.append(event)
            events.sort(key=lambda item: item.get("created_at_ms", 0),
             reverse=True)
            return events[:50]
        finally:
            await client.aclose()


    @app.get("/api/v1/admin/audit")
    async def admin_audit(request: Request, authorization: Optional[str] =
             Header(default=None), limit: int = 100):
        _require_admin(request, authorization)
        client = await _open_redis()
        if client is None:
            return []
        try:
            entries = await client.xrevrange("rhythmguard:audit", count=max(1,
             min(limit, 500)))
            return [{"stream_id": entry_id, **fields} for entry_id, fields in
             entries]
        finally:
            await client.aclose()


    @app.post("/api/v1/admin/sessions/{session_id}/action")
    async def admin_session_action(
        session_id: str,
        payload: AdminActionRequest,
        request: Request,
        authorization: Optional[str] = Header(default=None),
    ):
        _require_admin(request, authorization)
        allowed = {"require_mfa", "block", "restore", "acknowledge"}
        if payload.action not in allowed:
            raise HTTPException(status_code=422, detail=f"action must be one of:
             {', '.join(sorted(allowed))}")
        client = await _open_redis()
        if client is None:
            raise HTTPException(status_code=503, detail="Redis is unavailable")
        now_ms = int(time.time() * 1000)
        try:
            if payload.action == "require_mfa":
```

```
            await client.set(f"session:step_up:{session_id}",
             json.dumps({"source": "admin", "created_at_ms": now_ms}), ex=300)
        elif payload.action == "block":
            await client.delete(f"session:token:{session_id}")
            await client.set(f"session:blocked:{session_id}",
             json.dumps({"source": "admin", "created_at_ms": now_ms}), ex=900)
        elif payload.action == "restore":
            await client.delete(f"session:blocked:{session_id}")
            await client.delete(f"session:step_up:{session_id}")
        await _audit(client, {"event": "admin_action", "action":
         payload.action, "session_id": session_id, "note": payload.note,
         "created_at_ms": now_ms})
        return {"ok": True, "session_id": session_id, "action":
         payload.action, "created_at_ms": now_ms}
    finally:
        await client.aclose()


@app.post(
    "/api/v1/telemetry",
    status_code=status.HTTP_202_ACCEPTED,
    response_model=AcceptedResponse,
)
async def ingest_telemetry(payload: TelemetryRequest, request: Request):
    """Accept one or more micro-batches and hand them to Kafka
         immediately."""
    envelopes: List[TelemetryEnvelope] = payload if isinstance(payload, list)
         else [payload]
    if not envelopes:
        raise HTTPException(status_code=422, detail="at least one telemetry
         batch is required")

    publisher = await get_publisher(request)
    for envelope in envelopes:
        if not publisher.enqueue_nowait(as_dict(envelope)):
            raise HTTPException(status_code=503, detail="telemetry gateway
         queue is full")

    return {
        "accepted": len(envelopes),
        "queue_depth": publisher.queue_depth,
        "topic": publisher.topic,
    }


@app.get("/api/v1/alerts/recent")
async def recent_alerts():
    """Return recent persisted alerts before the dashboard opens its live
         stream."""
    if redis is None:
        return []
    client = redis.from_url(os.getenv("REDIS_URL",
         "redis://localhost:6379/0"), decode_responses=True)
```

```
    try:
        entries = await client.xrevrange("rhythmguard:alerts:history",
         count=100)
        alerts = []
        for entry_id, fields in entries:
            payload = fields.get("payload")
            if payload:
                try:
                    event = json.loads(payload)
                    event["stream_id"] = entry_id
                    alerts.append(event)
                except json.JSONDecodeError:
                    continue
        return alerts
    finally:
        await client.aclose()


@app.websocket("/ws/alerts")
async def alert_stream(websocket: WebSocket):
    """Forward worker risk events to the admin dashboard in real time."""
    await websocket.accept()
    if redis is None:
        await websocket.send_text(json.dumps({"event": "error", "message":
         "redis client is unavailable"}))
        await websocket.close(code=1011)
        return

    client = redis.from_url(os.getenv("REDIS_URL",
         "redis://localhost:6379/0"), decode_responses=True)
    pubsub = client.pubsub()
    await pubsub.subscribe("rhythmguard:alerts")
    try:
        while True:
            message = await
         pubsub.get_message(ignore_subscribe_messages=True, timeout=1.0)
            if message and message.get("type") == "message":
                await websocket.send_text(str(message["data"]))
            await asyncio.sleep(0.01)
    except WebSocketDisconnect:
        pass
    finally:
        try:
            await pubsub.unsubscribe("rhythmguard:alerts")
            await pubsub.aclose()
            await client.aclose()
        except Exception:
            pass

import math
from typing import List, Optional, Union
```

```python
from pydantic import BaseModel, Field, validator


FEATURE_COUNT = 4


class TelemetryEnvelope(BaseModel):
    """One client micro-batch sent to Kafka."""

    session_id: str = Field(..., min_length=1, max_length=128)
    telemetry: List[List[float]] = Field(..., min_items=1, max_items=200)
    sequence: int = Field(default=0, ge=0)
    client_timestamp_ms: Optional[float] = None
    event_types: Optional[List[str]] = None

    @validator("session_id")
    def session_id_is_safe(cls, value: str) -> str:
        if not value.strip() or any(ord(char) < 32 for char in value):
            raise ValueError("session_id must contain printable characters")
        return value.strip()

    @validator("telemetry")
    def telemetry_rows_are_finite(cls, value: List[List[float]]) ->
         List[List[float]]:
        for row in value:
            if len(row) != FEATURE_COUNT:
                raise ValueError(f"each telemetry row must have exactly
         {FEATURE_COUNT} features")
            if not all(math.isfinite(float(feature)) for feature in row):
                raise ValueError("telemetry features must be finite")
            if any(float(feature) < 0 for feature in row[:3]):
                raise ValueError("dwell, flight, and velocity must be non-
         negative")
        return value


TelemetryRequest = Union[TelemetryEnvelope, List[TelemetryEnvelope]]


class AcceptedResponse(BaseModel):
    accepted: int
    queue_depth: int
    topic: str


class AdminLoginRequest(BaseModel):
    password: str = Field(..., min_length=1, max_length=256)


class AdminActionRequest(BaseModel):
    action: str = Field(..., min_length=1, max_length=32)
    note: Optional[str] = Field(default=None, max_length=500)
```

Streaming Worker and Decision Engine

```python
import argparse
import asyncio
import json
import os
import math
from typing import Any, Dict, List, Optional, Tuple

import torch

from ml.model import BioSentinelLSTM, load_model
from worker.decision_engine import DecisionEngine

try:
    from aiokafka import AIOKafkaConsumer
except ImportError:  # pragma: no cover - dependency is installed in deployment
    AIOKafkaConsumer = None

try:
    import redis.asyncio as redis
except ImportError:  # pragma: no cover - dependency is installed in deployment
    redis = None


def decode_profile(raw: Any) -> Optional[Dict[str, Any]]:
    if raw is None:
        return None
    if isinstance(raw, bytes):
        raw = raw.decode("utf-8")
    if isinstance(raw, str):
        raw = json.loads(raw)
    if not isinstance(raw, dict):
        raise ValueError("Redis profile must be a JSON object")
    return raw


def profile_vectors(profile: Optional[Dict[str, Any]]) -> Tuple[List[float], List[float]]:
    profile = profile or {}
    baseline = profile.get("baseline", profile)
    means = baseline.get("mean", baseline.get("baseline_mean", [0, 0, 0, 0]))
    stds = baseline.get("std", baseline.get("baseline_std", [1, 1, 1, 1]))
    if len(means) != 4 or len(stds) != 4:
        raise ValueError("Redis profile mean/std must each contain four values")
    stds = [max(abs(float(value)), 1e-6) for value in stds]
    return [float(value) for value in means], stds
```

```
def fixed_window(rows: List[List[float]], means: List[float]) ->
        List[List[float]]:
    """Return exactly the last 20 rows, left-padded with the user's
        baseline."""
    cleaned = [[float(value) for value in row] for row in rows if len(row) ==
        4]
    cleaned = cleaned[-BioSentinelLSTM.sequence_length :]
    padding = [means[:] for _ in range(BioSentinelLSTM.sequence_length -
        len(cleaned))]
    return padding + cleaned


class RedisProfileStore:
    def __init__(self, url: str):
        if redis is None:
            raise RuntimeError("redis package is not installed")
        self.client = redis.from_url(url, decode_responses=False)

    async def get(self, session_id: str) -> Optional[Dict[str, Any]]:
        raw = await self.client.get(f"user:profile:{session_id}")
        return decode_profile(raw)

    async def close(self) -> None:
        await self.client.aclose()


def evaluate_payload(
    model: BioSentinelLSTM,
    payload: Dict[str, Any],
    profile: Optional[Dict[str, Any]],
    device: torch.device,
) -> float:
    if profile is None and hasattr(model, "profile_mean") and hasattr(model,
        "profile_std"):
        means = [float(value) for value in model.profile_mean]
        stds = [max(abs(float(value)), 1e-6) for value in model.profile_std]
    else:
        means, stds = profile_vectors(profile)
    rows = payload.get("telemetry", [])
    window = fixed_window(rows, means)
    features = torch.tensor([window], dtype=torch.float32, device=device)
    mean = torch.tensor(means, dtype=torch.float32, device=device)
    std = torch.tensor(stds, dtype=torch.float32, device=device)
    with torch.inference_mode():
        return float(model(features, baseline_mean=mean, baseline_std=std)
        [0].item())


def risk_label(score: float) -> str:
    if score > 0.7:
        return "critical"
    if score >= 0.4:
```

```
            return "suspicious"
        return "normal"


FEATURE_LABELS = [
    "Dwell time",
    "Flight time",
    "Mouse velocity",
    "Mouse acceleration",
]


def explain_payload(payload: Dict[str, Any], profile: Optional[Dict[str,
     Any]], model: BioSentinelLSTM) -> List[Dict[str, Any]]:
    """Create human-readable, baseline-relative evidence for the
     dashboard."""
    if profile is None and hasattr(model, "profile_mean") and hasattr(model,
     "profile_std"):
        means = [float(value) for value in model.profile_mean]
        stds = [max(abs(float(value)), 1e-6) for value in model.profile_std]
    else:
        means, stds = profile_vectors(profile)
    rows = [[float(value) for value in row] for row in
     payload.get("telemetry", []) if len(row) == 4][-20:]
    if not rows:
        return []
    factors = []
    for index, label in enumerate(FEATURE_LABELS):
        deviation = sum(abs(row[index] - means[index]) / stds[index] for row
     in rows) / len(rows)
        if deviation >= 1.5:
            factors.append({"feature": label, "deviation": round(deviation,
     2), "message": f"{label} differs from the saved baseline"})
    if len(rows) >= 5:
        for index, label in enumerate(("Dwell time", "Flight time")):
            mean_value = sum(row[index] for row in rows) / len(rows)
            variance = sum((row[index] - mean_value) ** 2 for row in rows) /
     len(rows)
            coefficient = math.sqrt(variance) / max(abs(mean_value), 1e-6)
            if coefficient < 0.05:
                factors.append({"feature": label, "deviation":
     round(coefficient, 3), "message": f"{label} is unusually regular;
     inspect for automation"})
    return factors[:6]


async def run_worker(
    bootstrap_servers: str,
    topic: str,
    group_id: str,
    redis_url: str,
    checkpoint: Optional[str] = None,
):
```

```
    if AIOKafkaConsumer is None:
        raise RuntimeError("aiokafka package is not installed")

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = load_model(checkpoint, device=device) if checkpoint else
         BioSentinelLSTM().to(device).eval()
    consumer = AIOKafkaConsumer(
        topic,
        bootstrap_servers=bootstrap_servers,
        group_id=group_id,
        enable_auto_commit=False,
        value_deserializer=lambda value: json.loads(value.decode("utf-8")),
    )
    profiles = RedisProfileStore(redis_url)
    decisions = DecisionEngine(profiles.client)
    await consumer.start()
    print(f"[worker] consuming {topic} with group {group_id}")
    try:
        async for message in consumer:
            payload = message.value
            session_id = str(payload.get("session_id", "")).strip()
            if not session_id:
                print("[worker] skipped message without session_id")
                await consumer.commit()
                continue

            try:
                await profiles.client.set("rhythmguard:worker:heartbeat",
         str(int(__import__("time").time() * 1000)), ex=30)
                profile = await profiles.get(session_id)
                score = evaluate_payload(model, payload, profile, device)
                factors = explain_payload(payload, profile, model)
                decision = await decisions.evaluate(session_id, score,
         factors)
                print(f"[worker] session={session_id} risk={score:.4f}
         status={decision.status}")
                if score > 0.7:
                    print(f"[CRITICAL] fraud-risk threshold exceeded for
         session {session_id}: {score:.4f}")
                await consumer.commit()
            except Exception as exc:
                # Do not commit malformed/inference-failed messages. Kafka
         can
                # redeliver them after the worker is repaired.
                print(f"[worker] processing failed for session={session_id}:
         {exc}")
    finally:
        await consumer.stop()
        await profiles.close()


def main():
```

```
    parser = argparse.ArgumentParser(description="RhythmGuard Kafka/Redis
        inference worker")
    parser.add_argument("--bootstrap-servers",
        default=os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092"))
    parser.add_argument("--topic", default=os.getenv("KAFKA_TOPIC",
        "telemetry-stream"))
    parser.add_argument("--group-id", default=os.getenv("KAFKA_GROUP_ID",
        "rhythmguard-inference"))
    parser.add_argument("--redis-url", default=os.getenv("REDIS_URL",
        "redis://localhost:6379/0"))
    parser.add_argument(
        "--checkpoint",
        default=os.getenv("RHYTHMGUARD_MODEL_CHECKPOINT",
         os.getenv("BIOSENTINEL_MODEL_CHECKPOINT")),
    )
    args = parser.parse_args()
    asyncio.run(run_worker(**vars(args)))


if __name__ == "__main__":
    main()

import json
import time
from dataclasses import dataclass
from typing import Any, Dict


ALERT_CHANNEL = "rhythmguard:alerts"
ALERT_HISTORY = "rhythmguard:alerts:history"
AUDIT_STREAM = "rhythmguard:audit"


@dataclass(frozen=True)
class RiskDecision:
    status: str
    action: str
    score: float
    session_id: str


class DecisionEngine:
    """Apply Phase 4 mitigation rules after model inference."""

    def __init__(self, redis_client: Any):
        self.redis = redis_client

    async def _persist_latest(self, decision: RiskDecision, reason: str,
         factors=None) -> None:
        event = {
            "event": "risk_evaluation",
            "status": decision.status,
            "action": decision.action,
```

```
            "score": decision.score,
            "session_id": decision.session_id,
            "reason": reason,
            "factors": factors or [],
            "created_at_ms": int(time.time() * 1000),
        }
        await self.redis.set(
            f"rhythmguard:session:{decision.session_id}:latest",
            json.dumps(event, separators=(",", ":")),
            ex=3600,
        )

    async def _publish_alert(self, decision: RiskDecision, reason: str,
     factors=None) -> None:
        event = {
            "event": "risk_alert",
            "status": decision.status,
            "action": decision.action,
            "score": decision.score,
            "session_id": decision.session_id,
            "reason": reason,
            "factors": factors or [],
            "created_at_ms": int(time.time() * 1000),
        }
        await self.redis.xadd(
            ALERT_HISTORY,
            {"payload": json.dumps(event, separators=(",", ":"))},
            maxlen=1_000,
            approximate=True,
        )
        await self.redis.publish(ALERT_CHANNEL, json.dumps(event, separators=
         (",", ":")))

    async def evaluate(self, session_id: str, score: float, factors=None) ->
     RiskDecision:
        now_ms = int(time.time() * 1000)

        if score < 0.4:
            decision = RiskDecision("normal", "continue_session", score,
         session_id)
            await self._persist_latest(decision, "risk score is below the
         step-up threshold", factors)
            # Redis Streams provide a lightweight audit sink for the local
            # deployment. A production deployment can replace this with a DB.
            await self.redis.xadd(
                AUDIT_STREAM,
                {"session_id": session_id, "score": str(score), "status":
         decision.status, "created_at_ms": str(now_ms)},
                maxlen=10_000,
                approximate=True,
            )
            return decision
```

```
        if score <= 0.7:
            decision = RiskDecision("suspicious", "require_step_up_mfa",
         score, session_id)
            await self.redis.set(
                f"session:step_up:{session_id}",
                json.dumps({"score": score, "created_at_ms": now_ms}),
                ex=300,
            )
            await self._persist_latest(decision, "risk score requires step-up
         authentication", factors)
            await self._publish_alert(decision, "risk score requires step-up
         authentication", factors)
            return decision

        decision = RiskDecision("critical", "block_session", score,
         session_id)
        # The token service can use these keys to reject future requests. The
        # delete is safe when no token has been provisioned for a smoke test.
        await self.redis.delete(f"session:token:{session_id}")
        await self.redis.set(
            f"session:blocked:{session_id}",
            json.dumps({"score": score, "created_at_ms": now_ms}),
            ex=900,
        )
        await self._persist_latest(decision, "critical fraud-risk threshold
         exceeded", factors)
        await self._publish_alert(decision, "critical fraud-risk threshold
         exceeded", factors)
        return decision
```

Evaluation Pipeline

```
"""Train and evaluate RhythmGuard without leaking windows between
        sessions."""

import argparse
import json
import random
from pathlib import Path
from typing import Dict, Iterable, List, Sequence, Tuple

import numpy as np
import torch
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
```

```
)
from torch.utils.data import DataLoader, TensorDataset, WeightedRandomSampler

from .dataset import load_labeled_dataset, profile_from_rows,
        read_feature_rows
from .model import BioSentinelLSTM, training_loop_placeholder


def split_session_files(
    paths: Sequence[str],
    validation_fraction: float = 0.2,
    test_fraction: float = 0.2,
    seed: int = 42,
) -> Tuple[List[str], List[str], List[str]]:
    """Split complete session files so rows from one session never cross
        splits."""
    if len(paths) < 3:
        raise ValueError(
            "at least 3 session files are required per class for
         train/validation/test splitting"
        )
    if validation_fraction <= 0 or test_fraction <= 0 or validation_fraction
         + test_fraction >= 1:
        raise ValueError("validation and test fractions must be positive and
         leave training data")

    shuffled = list(paths)
    random.Random(seed).shuffle(shuffled)
    test_count = max(1, round(len(shuffled) * test_fraction))
    validation_count = max(1, round(len(shuffled) * validation_fraction))
    if test_count + validation_count >= len(shuffled):
        test_count = validation_count = 1

    test = shuffled[:test_count]
    validation = shuffled[test_count : test_count + validation_count]
    train = shuffled[test_count + validation_count :]
    return train, validation, test


def grouped_split(
    normal_paths: Sequence[str],
    bot_paths: Sequence[str],
    validation_fraction: float,
    test_fraction: float,
    seed: int,
) -> Dict[str, Dict[str, List[str]]]:
    normal_train, normal_validation, normal_test = split_session_files(
        normal_paths, validation_fraction, test_fraction, seed
    )
    bot_train, bot_validation, bot_test = split_session_files(
        bot_paths, validation_fraction, test_fraction, seed + 1
    )
```

```
    return {
        "train": {"normal": normal_train, "bot": bot_train},
        "validation": {"normal": normal_validation, "bot": bot_validation},
        "test": {"normal": normal_test, "bot": bot_test},
    }


def normalized_dataset(
    normal_paths: Iterable[str],
    bot_paths: Iterable[str],
    mean: torch.Tensor,
    std: torch.Tensor,
    stride: int,
) -> TensorDataset:
    dataset = load_labeled_dataset(normal_paths, bot_paths, stride=stride)
    features, labels = dataset.tensors
    return TensorDataset((features - mean) / std, labels)


def predict(model: BioSentinelLSTM, dataset: TensorDataset, batch_size: int)
        -> Tuple[np.ndarray, np.ndarray]:
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=False)
    scores: List[np.ndarray] = []
    labels: List[np.ndarray] = []
    model.eval()
    with torch.inference_mode():
        for features, batch_labels in loader:
            scores.append(model(features).cpu().numpy())
            labels.append(batch_labels.cpu().numpy())
    return np.concatenate(labels), np.concatenate(scores)


def metrics_at_threshold(labels: np.ndarray, scores: np.ndarray, threshold:
        float) -> Dict[str, object]:
    predictions = (scores >= threshold).astype(np.int64)
    unique_labels = np.unique(labels)
    auc = float(roc_auc_score(labels, scores)) if len(unique_labels) == 2
        else None
    return {
        "threshold": float(threshold),
        "samples": int(len(labels)),
        "accuracy": float(accuracy_score(labels, predictions)),
        "precision": float(precision_score(labels, predictions,
        zero_division=0)),
        "recall": float(recall_score(labels, predictions, zero_division=0)),
        "f1": float(f1_score(labels, predictions, zero_division=0)),
        "roc_auc": auc,
        "confusion_matrix": confusion_matrix(labels, predictions, labels=[0,
        1]).tolist(),
    }
```

```
def tune_threshold(labels: np.ndarray, scores: np.ndarray) -> Tuple[float,
    Dict[str, object]]:
    """Choose the threshold with the best validation F1, never using test
    labels."""
    candidates = np.unique(np.concatenate(([0.0], scores, [1.0])))
    results = [metrics_at_threshold(labels, scores, float(candidate)) for
    candidate in candidates]
    best = max(results, key=lambda result: (result["f1"],
    result["precision"], result["threshold"]))
    return float(best["threshold"]), best


def load_profile(split: Dict[str, List[str]]) -> Tuple[torch.Tensor,
    torch.Tensor]:
    normal_rows = [row for path in split["normal"] for row in
    read_feature_rows(path)]
    if not normal_rows:
        raise ValueError("training split contains no normal telemetry rows
        for profile calculation")
    return profile_from_rows(normal_rows)


def balanced_loader(dataset: TensorDataset, batch_size: int) -> DataLoader:
    """Balance normal/bot windows without duplicating validation or test
    data."""
    labels = dataset.tensors[1].long()
    counts = torch.bincount(labels, minlength=2).float().clamp_min(1.0)
    sample_weights = 1.0 / counts[labels]
    sampler = WeightedRandomSampler(
        weights=sample_weights,
        num_samples=len(dataset),
        replacement=True,
    )
    return DataLoader(dataset, batch_size=batch_size, sampler=sampler)


def main():
    parser = argparse.ArgumentParser(description="Leakage-safe RhythmGuard
    model evaluation")
    parser.add_argument("--normal", nargs="+", required=True, help="normal
    session JSON files")
    parser.add_argument("--bot", nargs="+", required=True, help="bot session
    JSON files")
    parser.add_argument("--validation-fraction", type=float, default=0.2)
    parser.add_argument("--test-fraction", type=float, default=0.2)
    parser.add_argument("--stride", type=int, default=5)
    parser.add_argument("--epochs", type=int, default=10)
    parser.add_argument("--batch-size", type=int, default=32)
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--output",
    default="artifacts/rhythmguard_evaluated.pt")
    parser.add_argument("--report",
    default="artifacts/evaluation_report.json")
    args = parser.parse_args()
```

```
random.seed(args.seed)
np.random.seed(args.seed)
torch.manual_seed(args.seed)
splits = grouped_split(
    args.normal,
    args.bot,
    args.validation_fraction,
    args.test_fraction,
    args.seed,
)
mean, std = load_profile(splits["train"])
train_dataset = normalized_dataset(
    splits["train"]["normal"], splits["train"]["bot"], mean, std,
     args.stride
)
validation_dataset = normalized_dataset(
    splits["validation"]["normal"], splits["validation"]["bot"], mean,
     std, args.stride
)
test_dataset = normalized_dataset(
    splits["test"]["normal"], splits["test"]["bot"], mean, std,
     args.stride
)

model = training_loop_placeholder(
    BioSentinelLSTM(), balanced_loader(train_dataset, args.batch_size),
     epochs=args.epochs
)
validation_labels, validation_scores = predict(model, validation_dataset,
     args.batch_size)
threshold, validation_metrics = tune_threshold(validation_labels,
     validation_scores)
test_labels, test_scores = predict(model, test_dataset, args.batch_size)
test_metrics = metrics_at_threshold(test_labels, test_scores, threshold)

report = {
    "seed": args.seed,
    "split_files": splits,
    "window_counts": {
        "train": len(train_dataset),
        "validation": len(validation_dataset),
        "test": len(test_dataset),
    },
    "class_counts": {
        split_name: {
            "normal": int((dataset.tensors[1] == 0).sum().item()),
            "bot": int((dataset.tensors[1] == 1).sum().item()),
        }
        for split_name, dataset in (
            ("train", train_dataset),
            ("validation", validation_dataset),
```

```
                ("test", test_dataset),
            )
        },
        "threshold_selection": "maximum validation F1; test labels were not
         used",
        "validation": validation_metrics,
        "test": test_metrics,
        "profile": {"mean": mean.tolist(), "std": std.tolist()},
    }
    output_path = Path(args.output)
    report_path = Path(args.report)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    report_path.parent.mkdir(parents=True, exist_ok=True)
    torch.save(
        {
            "model_state_dict": model.state_dict(),
            "sequence_length": BioSentinelLSTM.sequence_length,
            "feature_count": BioSentinelLSTM.feature_count,
            "profile": {"mean": mean.tolist(), "std": std.tolist()},
            "decision_threshold": threshold,
        },
        output_path,
    )
    report_path.write_text(json.dumps(report, indent=2), encoding="utf-8")
    print(json.dumps({"validation": validation_metrics, "test":
         test_metrics}, indent=2))
    print(f"saved evaluated checkpoint to {output_path}")
    print(f"saved evaluation report to {report_path}")


if __name__ == "__main__":
    main()
```

References and Further Reading

1. PyTorch documentation, https://pytorch.org/docs/.
2. FastAPI documentation, https://fastapi.tiangolo.com/.
3. Apache Kafka documentation, https://kafka.apache.org/documentation/.
4. Redis documentation, https://redis.io/docs/.
5. OWASP Application Security Verification Standard, https://owasp.org/www-project-application-security-verification-standard/.
6. NIST Digital Identity Guidelines, https://pages.nist.gov/800-63-4/.